COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	9/20/26	**DATE SUBMITTED:**	9/21/26
NAME:	Mike Justin A. Cabangon	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 **SORTING ALGORITHMS Pt2**

1. PROCEDURE OUTPUT

ILO A

CODE: Main

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

const int SIZE = 100;

int main() {
    int arr[SIZE];
    srand(time(0));
    for (int i = 0; i < SIZE; i++) {
        arr[i] = rand() % 1000 + 1;
        std::cout << arr[i] << " ";
    }
}
```

OUTPUT

```
873 520 309 387 47 560 172 267 27 868 77 895 694 624
835 859 601 911 376 536 338 621 266 176 859 261 556 217
683 149 210 176 946 969 988 567 55 893 415 458 867 71
408 231 603 532 739 605 626 706 883 942 158 589 835 839
156 854 44 79 158 841 840 718 884 574 14 865 465 443 95
595 551 544 369 342 413 370 257 426 808 412 414 301 352
811 676 964 157 1 183 912 657 116 464 757 968 152 829
151
```

OBSERVATION
The program prints 100 random, unordered values between 1 and 1000 (with some repeats), and the array is different on each run.

Table 8-1 Array of Values for Sort Algorithm Testing

CODE: Main

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingalgo.h"

const int size = 100;

void copyArray(int source[], int destination[], int size);
void displayArr(int array[], int size);

int main(){

    int original[size];
    int shellArray[size];

    srand(time(0));

    for (int i = 0; i < size; i++)
    {
        original[i] = rand() % 1000 + 1;
    }
    copyArray(original, shellArray, size);

    std::cout << "Original Unsorted Array:" << std::endl;
    displayArr(original, size);

    shellSort(shellArray, size);

    std::cout << std::endl;
    std::cout << "Shell Sort:" << std::endl;
```

```
    displayArr(shellArray, size);

    return 0;
}
void copyArray(int source[], int destination[], int size)
{
    for (int i = 0; i < size; i++)
    {
        destination[i] = source[i];
    }
}

void displayArr (int array[], int size){
    for(int i = 0; i < size; i++){
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;
}
```

CODE: Header

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <iostream>
#include <utility>

template <typename T>
void shellSort(T arr[], int arrSize) {
    int n = arrSize;

    // Start with a large gap, then reduce it step by step
    for (int gap = n / 2; gap > 0; gap /= 2) {

        // Perform a "gapped" insertion sort for this gap size
        for (int i = gap; i < n; i++) {

            // Current element to be placed correctly
            T temp = arr[i];
            int j = i;

            // Shift elements that are greater than temp to make space
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }

            // Place temp in its correct location
            arr[j] = temp;
        }
    }
}

#endif
```

OUTPUT

```
Original Unsorted Array:
269 974 592 249 300 359 89 325 648 638 95 167 220 603 244 73 746 632 12 497 447 708 118
861 368 826 277 264 909 489 464 62 764 149 887 436 113 38 231 155 100 655 951 631 476 433
36 187 179 665 874 623 898 944 796 317 943 170 454 88 123 858 464 382 81 323 862 992 972
873 246 31 192 730 865 225 124 431 366 780 886 207 300 27 543 758 638 343 368 337 644 39
918 854 263 498 203 434 756 852

Shell Sort:
12 27 31 36 38 39 62 73 81 88 89 95 100 113 118 123 124 149 155 167 170 179 187 192 203
207 220 225 231 244 246 249 263 264 269 277 300 300 317 323 325 337 343 359 366 368 368
382 431 433 434 436 447 454 464 464 476 489 497 498 543 592 603 623 631 632 638 638 644
648 655 665 708 730 746 756 758 764 780 796 826 852 854 858 861 862 865 873 874 886 887
898 909 918 943 944 951 972 974 992
```

OBSERVATION

The program prints the 100 random values unsorted, then the same values in ascending order after Shell sort, with nothing added or lost.

Table 8-2 Shell Sort Technique

CODE: Main

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingalgo.h"

const int size = 100;

void copyArray(int source[], int destination[], int size);
void displayArr(int array[], int size);

int main(){

    int original[size];
    int mergeArray[size];

    srand(time(0));

    for (int i = 0; i < size; i++)
    {
        original[i] = rand() % 1000 + 1;
    }
    copyArray(original, mergeArray, size);

    std::cout << "Original Unsorted Array:" << std::endl;
    displayArr(original, size);

    mergeSort(mergeArray, 0, size - 1);

    std::cout << std::endl;
    std::cout << "Merge Sort:" << std::endl;
    displayArr(mergeArray, size);

    return 0;
}
void copyArray(int source[], int destination[], int size)
{
    for (int i = 0; i < size; i++)
    {
        destination[i] = source[i];
    }
}

void displayArr (int array[], int size){
    for(int i = 0; i < size; i++){
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;
}
```

CODE: Header

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <iostream>
#include <utility>

// Merges two subarrays of arr[].
// First subarray is arr[left..mid]
// Second subarray is arr[mid+1..right]
template <typename T>
void merge(T arr[], int left, int mid, int right){

    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temp vectors
    T* L = new T[n1];
    T* R = new T[n2];

    // Copy data to temp vectors L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0;
    int k = left;

    // Merge the temp vectors back
    // into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
```

```
41      }
42
43      // Copy the remaining elements of L[],
44      // if there are any
45      while (i < n1) {
46          arr[k] = L[i];
47          i++;
48          k++;
49      }
50
51      // Copy the remaining elements of R[],
52      // if there are any
53      while (j < n2) {
54          arr[k] = R[j];
55          j++;
56          k++;
57      }
58
59      delete[] L;
60      delete[] R;
61  }
62
63  // begin is for left index and end is right index
64  // of the sub-array of arr to be sorted
65  template <typename T>
66  void mergeSort(T arr[], int left, int right){
67
68      if (left >= right)
69          return;
70
71      int mid = left + (right - left) / 2;
72      mergeSort(arr, left, mid);
73      mergeSort(arr, mid + 1, right);
74      merge(arr, left, mid, right);
75  }
76
77  #endif
```

OUTPUT

```
Original Unsorted Array:
973 546 577 387 556 728 388 616 715 611 424 160 610 33 449 589 35 807 375 143 565 62 306 465 176
341 219 356 123 893 93 626 267 641 412 990 869 204 413 442 174 160 422 841 833 196 159 830 537 517
322 444 383 241 342 360 741 960 786 4 892 932 568 310 219 66 344 112 289 224 519 816 774 490 335 99
613 144 465 430 836 931 120 557 519 656 328 295 522 299 695 810 261 720 923 755 541 877 324 625

Merge Sort:
4 33 35 62 66 93 99 112 120 123 143 144 159 160 160 174 176 196 204 219 219 224 241 261 267 289 295
299 306 310 322 324 328 335 341 342 344 356 360 375 383 387 388 412 413 422 424 430 442 444 449 465
465 490 517 519 519 522 537 541 546 556 557 565 568 577 589 610 611 613 616 625 626 641 656 695 715
720 728 741 755 774 786 807 810 816 830 833 836 841 869 877 892 893 923 931 932 960 973 990
```

OBSERVATION

The code splits the array in half again and again, then joins the small pieces back together in order. The output shows the same numbers, now sorted from smallest to largest.

Table 8-3 Merge Sort Algorithm

CODE: Main

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingalgo.h"

const int size = 100;

void copyArray(int source[], int destination[], int size);
void displayArr(int array[], int size);

int main(){

    int original[size];
    int quickArray[size];

    srand(time(0));

    for (int i = 0; i < size; i++)
    {
        original[i] = rand() % 1000 + 1;
    }
    copyArray(original, quickArray, size);

    std::cout << "Original Unsorted Array:" << std::endl;
    displayArr(original, size);

    quickSort(quickArray, 0, size - 1);

    std::cout << std::endl;
    std::cout << "Quick Sort:" << std::endl;
    displayArr(quickArray, size);

    return 0;
}
void copyArray(int source[], int destination[], int size)
{
    for (int i = 0; i < size; i++)
    {
        destination[i] = source[i];
    }
}

void displayArr (int array[], int size){
    for(int i = 0; i < size; i++){
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;
}
```

CODE: Header

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <iostream>
#include <utility>

template <typename T>
int partition(T arr[], int low, int high) {

    // choose the pivot
    T pivot = arr[high];

    // undex of smaller element and indicates
    // the right position of pivot found so far
    int i = low - 1;

    // Traverse arr[low..high] and move all smaller
    // elements on left side. Elements from low to
    // i are smaller after every iteration
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    // move pivot after smaller elements and
    // return its position
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

// the QuickSort function implementation
template <typename T>
void quickSort(T arr[], int low, int high) {

    if (low < high) {

        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);

        // recursion calls for smaller elements
        // and greater or equals elements
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

#endif

```

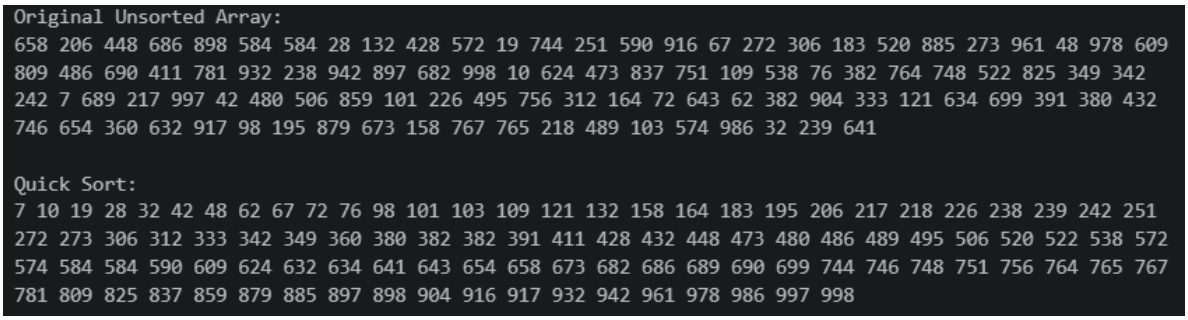
OUTPUT

OBSERVATION
The program prints the 100 random values unsorted, then the same values sorted from smallest to largest. The code picks one number (the pivot), puts smaller numbers on its left and bigger ones on its right, then repeats this on each side.

Table 8-4 Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1
Yes. After partition, the pivot is already in the right place, and the left and right lists are separate. So we can sort each one with any other algorithm, like shell sort. In the example, partition runs once, then shell sort sorts each side, and the whole array ends up sorted.
PROBLEM 2
Merge sort is the fastest for this array because it made the fewest comparisons (42, against 47 for shell and quick sort). Merge sort and quick sort are O(N log N) because they keep cutting the list in half (about log N times), and each cut level looks at all N numbers once. So the total work is N x log N.

CODE: Main

```cpp
#include <iostream>
#include "supp.h"

void displayArr(int array[], int size);

int main(){

    int arr[] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};
    const int size = 15;

    std::cout << "Original Array:" << std::endl;
    displayArr(arr, size);

    // quick sort's partition step: pivot lands in its final position
    int pi = partition(arr, 0, size - 1);

    std::cout << std::endl;
    std::cout << "After Partition (pivot " << arr[pi] << "):" << std::endl;
    displayArr(arr, size);

    // sort each sub list with shell sort instead of quick sort
    shellSort(arr, pi);                     // left sub list
    shellSort(arr + pi + 1, size - pi - 1);  // right sub list

    std::cout << std::endl;
    std::cout << "After Shell Sort on Both Sub Lists:" << std::endl;
    displayArr(arr, size);

    return 0;
}

void displayArr (int array[], int size){
    for(int i = 0; i < size; i++){
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;
}
```

CODE: Header

```
#ifndef SUPP_H
#define SUPP_H

#include <iostream>
#include <utility>

template <typename T>
void shellSort(T arr[], int arrSize) {
    int n = arrSize;

    // Start with a large gap, then reduce it step by step
    for (int gap = n / 2; gap > 0; gap /= 2) {

        // Perform a "gapped" insertion sort for this gap size
        for (int i = gap; i < n; i++) {

            // Current element to be placed correctly
            T temp = arr[i];
            int j = i;

            // Shift elements that are greater than temp to make space
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }

            // Place temp in its correct location
            arr[j] = temp;
        }
    }
}
```

```cpp

template <typename T>
int partition(T arr[], int low, int high) {

    // choose the pivot
    T pivot = arr[high];

    // index of smaller element and indicates
    // the right position of pivot found so far
    int i = low - 1;

    // Traverse arr[low..high] and move all smaller
    // elements on left side. Elements from low to
    // i are smaller after every iteration
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    // move pivot after smaller elements and
    // return its position
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

#endif

```

OUTPUT

```
Original Array:
4 34 29 48 53 87 12 30 44 25 93 67 43 19 74

After Partition (pivot 74):
4 34 29 48 53 12 30 44 25 67 43 19 74 87 93

After Shell Sort on Both Sub Lists:
4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
```

OBSERVATION
The array is split around the pivot 74, then shell sort sorts each side, and the final array is in order from smallest to largest

CONCLUSION
I learned how shell sort, merge sort, and quick sort work and how they differ from each other. Shell sort sorts items that are far apart first, then closes the gap step by step. Merge sort splits the list in half and joins the sorted halves back together. Quick sort picks a pivot, puts smaller numbers on its left and bigger on its right, then repeats on each side We made an array of 100 random values, sorted, and printed the results. All three algorithms produced the same sorted list, so each one works correctly.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.